

PRESENTATION PROBLEMS

Instructor: Orhan Özgüner

© Özgüner @CWRU, 2025

Presentation 1

You are working for a company that designs and manufactures made-to-order radio antenna equipment, and want to create an automated method of designing transmitters to fill client orders. Clients will specify a given power factor t , and you must connect together amplifier chips to exactly match that value. Each chip has a stated power factor (at least 1), and the factor of multiple chips connected together is the *product* of their individual factors. The preprinted circuit boards in stock have slots for three chips each, and all three slots must be filled to complete the circuit.

Formally, if C is an unsorted list of all the chips in stock, where $C[x]$ is the amplification factor of chip x , you need to efficiently find all possible unique triplets $(C[i], C[j], C[k])$, where $i \neq j \neq k$ and $C[i] \times C[j] \times C[k] = t$. (Reorderings, such as $(C[i], C[j], C[k])$ vs $(C[k], C[j], C[i])$, are not considered unique.) Develop an efficient algorithm to solve this problem.

Example:

Input:

- $C = [2, 3, 6, 12, 24, 1, 4]$
- $t = 48$

Output: $[(1, 4, 12), (1, 2, 24), (2, 4, 6)]$

Presentation 1 Solution

Pseudocode

Algorithm 1 Finding All Triplets With Target Power

```
1: procedure designAmplifiers( $C, t$ )
2:   MERGESORT( $C$ )                                ▷ Sort in increasing order
3:    $result \leftarrow []$ 
4:    $i = 0$ 
5:   while  $i < \text{len}(C) - 2$  do                      ▷ Outer Loop (OL)
6:      $target \leftarrow \frac{t}{C[i]}$ 
7:      $low \leftarrow i + 1$ 
8:      $high \leftarrow \text{length}(C) - 1$ 
9:     while  $low < high$  do                            ▷ Inner Loop (IL)
10:      if  $C[low] * C[high] = target$  then
11:         $result.append([C[i], C[low], C[high]])$ 
12:         $low \leftarrow low + 1$ 
13:         $high \leftarrow high - 1$ 
14:      else if  $C[low] * C[high] < target$  then
15:         $low \leftarrow low + 1$ 
16:      else
17:         $high \leftarrow high - 1$ 
18:      end if
19:    end while
20:     $i \leftarrow i + 1$ 
21:  end while
22:  return  $result$ 
23: end procedure
```

Proof of Correctness

Outer Loop Invariant:

1. $result$ stores all valid triplets $(C[p], C[q], C[r])$ s.t. $p < i$ and $p < q < r$.
2. The triplets collected do not include any duplicates.

Poof:

- **Initialization:** Before the first iteration, $i = 0$. Therefore, elements with index $j < 0$ do not exist, and there are no valid triplets, so statement (1) of the L.I. holds. Also, $result$ stores no triplets, so statement (2) of the L.I. holds.
- **Maintenance:** Assume that the L.I. holds for some arbitrary iteration. First, assume that $C[i]$ will be the first element in the valid triplet being searched for.

Then, *target* is initialized to the actual target, t , divided by $C[i]$. That way, if two other elements multiply to equal *target*, we have found a new valid triplet. The inner loop finds all pairs $(C[low], C[high])$ s.t. $C[low] * C[high] = target$, ensuring that every possible valid triplet involving $C[i]$ as the first element is found. Once i is incremented, statement (1) of the L.I. will be true.

The inner loop also guarantees that no triplet examined before this iteration, is examined again. Therefore, no triplet can be included twice, and statement (2) of the L.I. must hold.

- **Termination:** The outer loop terminates when $i = len(C) - 2$. Substituting this fact into the L.I. tells us *result* stores all valid triplets $(C[p], C[q], C[r])$ s.t. $p < q < r$. There exist no possible triplets with $p \geq len(C) - 2$, because there is no valid value for r to take on and still be in bounds. Therefore, *result* stores all the valid triplets from C .

Statement (2) of the L.I. tells us that we did not collect any duplicates when traversing through the entire list, thus we have achieved the goal of this program.

Inner Loop Invariant:

1. *result* contains all valid triplets $(C[i], C[low'], C[high'])$ s.t. $i < low' < low$ and $high < high' < len(C)$.
2. All valid triplets $(C[i], C[low'], C[high'])$ s.t. $i < low' < low$ and $high < high' < len(C)$ have $i \neq low' \neq high'$.

Proof

- **Initialization:** Before the first iteration, $low = i + 1$ and $high = len(C) - 1$. There are no indices $i < low' < i + 1$ or $len(C) - 1 > high' > high$, so the L.I. is vacuously true.
- **Maintenance:** Assume that the loop invariant holds. We must consider the following three cases:

1. Case where we have found a valid triplet
 - If the product $C[low] * C[high] = \frac{t}{C[i]}$, the triplet $(C[i], C[low], C[high])$ is valid and added to the result list. Since $high > low$, and low started $> i$, indices i , low , and $high$ are distinct. Thus, the L.I. is maintained.
2. $C[low] * C[high] < target$
 - We need to remind ourselves of two important properties of our algorithm: C is sorted in increasing order, and $high$ will never *increase* in value. So, any other value that we have not already checked, that is multiplied with $C[low]$, will result in a product that is $< target$. Thus, low will not be a part of any future triplets, so we can increment low to continue looking for pairs. This proves Clause (1) of the L.I. Additionally, since we have not added any triplets, we also maintain Clause (2) of the L.I.

3. $C[low] * C[high] < target$

- We need to remind ourselves of two important properties of our algorithm: C is sorted in increasing order, and low will never *decrease* in value. So, any other value that we have not already checked, that is multiplied with $C[high]$, will result in a product that is $< target$. Thus, $high$ will not be a part of any future triplets, so we can decrement $high$ to continue looking for pairs. This proves Clause (1) of the L.I. Additionally, since we have not added any triplets, we also maintain Clause (2) of the L.I.
- **Termination:** The inner loop terminates when $low \geq high$, meaning all valid pairs for the current i have been checked. The invariant ensures that all triplets involving $C[i]$ are distinct and valid, and no duplicate triplets have been added to the result list. Thus, the invariant guarantees that all valid triplets have been added during the inner loop and are distinct.

Runtime Analysis

First, we must consider the runtime of sorting C using *mergesort* to be $O(n \log(n))$.

Now, let's consider the outer loop. In each iteration of the outer loop, i is incremented by 1. Thus, the outer loop runs $n - 2$ times.

In each iteration of the outer loop, we initialize certain variables and run the inner loop. The inner loop runs while $low < high$. This means that in each iteration of the outer loop (for a given i), the two-pointer search is executed once, which processes the entire subarray of size $n - i - 1$. Thus, the inner loop runs in linear time relative to the size of the subarray. Thus, the nested loop structure runs in $O(n^2)$.

We reach the final run time of

$$T(n) = O(n^2)$$

Presentation 2

You are working for a telecommunications company, that has been contracted to provide wireless internet for customers along a long, straight stretch of rural highway. The location of each customer is given in miles starting from the beginning of the highway, in an array B . Your company has already purchased a series of existing radio towers along the highway, with locations given in an array T . Now, you need to purchase the antenna equipment to install on the towers. Antennas are priced according to the *distance* a customer can be from the nearest tower and still get a signal (either forward or backward along the highway). All the antennas need to be the same so that they can be purchased in bulk. It is OK if a customer is in range of two or more towers, but each customer has to be in range of at least one.

Neither C nor T is sorted, but neither C nor T will contain duplicate values.

What is the minimum transmitter range r , required so that every customer in C is in range of at least one tower? Develop an efficient algorithm to calculate this value.

Example:

Input:

- $C = [4, 2, 6, 1, 5, 9, 3, 7]$
- $T = [2, 9, 6]$

Output: $r = 2$

Presentation 2 Solution

Pseudocode

Algorithm 2 Minimum Antenna Radius

```
procedure FindMinRadius( $C, T$ )
  sort  $C$ 
  sort  $T$ 
   $minRadius \leftarrow 0$ 
   $i \leftarrow 0$ 
   $j \leftarrow 0$ 
  while  $i < C.length$  do
     $nextT \leftarrow j$ 
    while  $nextT + 1 < T.length$  and  $|C[i] - T[nextT]| \geq |C[i] - T[nextT + 1]|$  do
       $nextT \leftarrow nextT + 1$ 
    end while
     $j \leftarrow nextT$ 
     $distance \leftarrow |C[i] - T[j]|$ 
     $minRadius \leftarrow \max\{minRadius, distance\}$ 
     $i \leftarrow i + 1$ 
  end while
  return  $minRadius$ 
end procedure
```

Proof of Correctness

Outer Loop Invariant:

- $minRadius$ = the minimum transmission radius if only considering customers $C[0..i - 1]$
- $T[j]$ is the rightmost closest tower to customer $C[i - 1]$: i.e. $j = \max_k \{\arg \min \{|C[i - 1] - T[k]| \}\}$

Proof

- **Initialization:** Before the loop, $i = 0$. Therefore, the subarray $C[0.. - 1]$ does not exist, so the L.I. is vacuously true.
- **Maintenance:** Assume the L.I. is true for some arbitrary iteration of the loop. In order to prove the L.I., consider 3 cases:
 - $T[j] < C[i]$
Since array T and C are sorted with unique values, $T[0..j - 1] < T[j] < C[i]$ which means that any towers from $T[0..j - 1]$ are *not* the closest to customer $C[i]$. There is still a chance that $T[j]$ is the closest tower for $C[i]$, but there's also a chance that the closest tower to $C[i]$ is further down the highway.

We've proven below that at the end of the inner loop $T[nextT]$ will be the rightmost closest tower to customer $C[i]$ out of all towers in $T[j...T.length - 1]$. Since we reassign j to the value of $nextT$ after the inner loop, that means statement (2) of the L.I. will be true at the next iteration.

Since statement (2) is true, that must mean *distance* is a minimum. If this minimum distance for customer $C[i]$ is larger than the minimum distance for customers $C[0..i - 1]$, then we must update *minRadius*. Once this is performed, statement (1) of the L.I. also holds at the next iteration.

– $T[j] > C[i]$

Since array T is sorted, none of the towers $T[j + 1...T.length - 1]$ will be closer to customer $C[i]$, since $C[i] < T[j] < T[j + 1...T.length - 1]$.

Let $T[j] - C[i - 1] = d$ be the minimum transmission distance for customer $C[i]$. Since array C is sorted, $C[i - 1] < C[i]$. Therefore, $T[j] - C[i] < T[j] - C[i - 1] = d$. Because of statement (2) of the L.I., $C[i - 1] - T[j - 1] \geq d$, otherwise $T[j]$ wouldn't be the leftmost closest tower. Again, since array C is sorted, $C[i] - T[j - 1] > C[i - 1] - T[j - 1] \geq d$. Since array T is sorted, this means that no towers $T[0...j - 1]$ are the rightmost closest tower to customer $C[i]$.

Therefore, the rightmost closest tower to customer $C[i]$ must be $T[j]$, so statement (2) of the L.I. will hold at the next iteration. Since the $T[j] - C[i] < T[j] - C[i - 1]$, that means we did not encounter a new largest minimum transmission distance. Therefore statement (1) of the L.I. will hold at the next iteration.

– $C[j] = T[i]$

The distance between customer $C[i]$ and the rightmost closest tower from the previous iteration is 0 (as small as possible). Since elements in array T are unique, we don't change j , which ensures statement (2) of the L.I. will hold at the next iteration.

Since *minRadius* was initialized to 0, we can guarantee that we have not encountered a new largest monitoring radius. *minRadius* won't change value, so statement (1) of the L.I. holds at the next iteration.

- **Termination:** At termination, $i \geq C.length$. Since i increments at each step in the loop, it can be said further that $i = C.length$. Therefore, the L.I. tells us that *minRadius* is the minimum monitoring radius if only considering customers $C[0...n]$, which is the entire array. Therefore, the goal of the loop has been achieved.

Inner Loop Invariant

Out of towers $T[j...nextT - 1]$, tower $T[nextT]$ is the rightmost closest tower to customer $C[i]$, i.e. $nextT = \max_k \{\arg \min_k \{|d - T[k]|\}\}$ for $j \leq k \leq nextT - 1$

Proof

- Initialization: Before the loop, $nextT = j$. There exists no k s.t. $j \leq k \leq j - 1$. Therefore, the L.I. is vacuously true.

- **Maintenance:** Assume the L.I. holds at some arbitrary iteration. Given the condition of the **while** loop, there are 2 cases:
 - $|C[i] - T[nextT]| > |C[i] - T[nextT + 1]|$
 In this case, the tower directly to the right of $T[nextT]$ is actually closer to customer $C[i]$. Therefore, after incrementing $nextT$, the L.I. will hold at the next iteration.
 - $|C[i] - T[nextT]| = |C[i] - T[nextT + 1]|$
 In this case, the towers $T[nextT]$ and $T[nextT + 1]$ are the same distance away from customer $C[i]$. Since array T is sorted, $T[nextT] < T[nextT + 1]$. Therefore, $T[nextT + 1]$ is the rightmost closest tower according to the definition in the L.I.. After incrementing $nextT$, the L.I. will hold at the next iteration.
- **Termination:** Remember, the goal of the inner loop was to show that out of towers $T[j...T.length - 1]$, $nextT$ will point to the rightmost closest tower after execution. At termination there are two possible cases:
 - $nextT + 1 \geq T.length$
 Since $nextT$ increments at each iteration, we can instead say that $nextT + 1 = T.length$. At this point, $T[T.length - 1]$ is the rightmost closest tower to customer $C[i]$, which makes sense since there are no more towers to the right of $T[T.length - 1]$. Therefore, the goal of the loop has been achieved.
 - $|C[i] - T[nextT]| < |C[i] - T[nextT + 1]|$
 The L.I. tells us that $T[nextT]$ is the rightmost closest tower out of the subarray $T[j...nextT - 1]$. Also, since T is sorted in increasing order, this means that no towers to the right of $T[nextT]$ have a closer distance. Therefore, the goal of the loop has been achieved.

Runtime Analysis

Let $m = C.length$ and $n = T.length$. First, note that both arrays are sorted. If we assume the arrays have been sorted efficiently with a method such as mergesort or quicksort, then the sorting operation will take $\Theta(n \log n + m \log m)$ time.

Next, note that the number of iterations of the outer loop is m since we iterate over all the customers. Since j iterates through the list of towers and is always increasing, that means the number of *total* iterations of the inner loop during the entire algorithm is n . We can guarantee this because $nextT + 1$ is bounds-checked at each iteration. Given that the rest of the instructions in the outer loop execute in constant time, combining all the iterations of the outer loop gives $\Theta(m + n)$ time.

The sorting portion of the algorithm dominates, so the final runtime is

$$\Theta(n \log n + m \log m)$$